

```
<!--Análisis De Algoritmos-->
```

Lowest Common Ancestor {

```
<Método=" Binary Lifting - Divide y Vencerás"/>
```

```
<Por="
```

```
Ismael Gándara Cornejo,
```

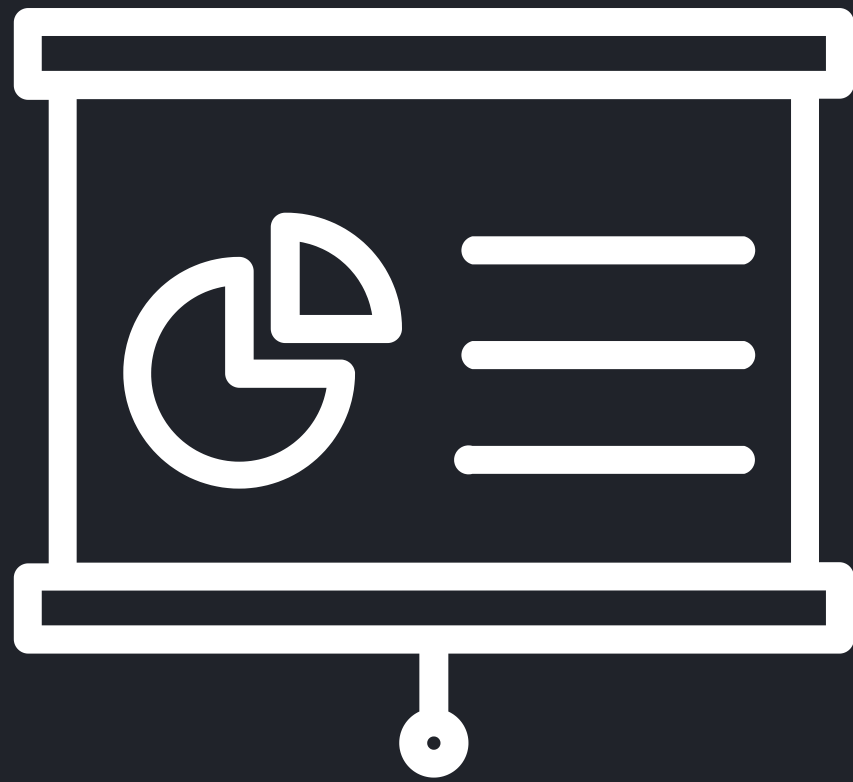
```
Ángel Miguel Villalvazo Vázquez
```

```
"/>
```

```
}
```



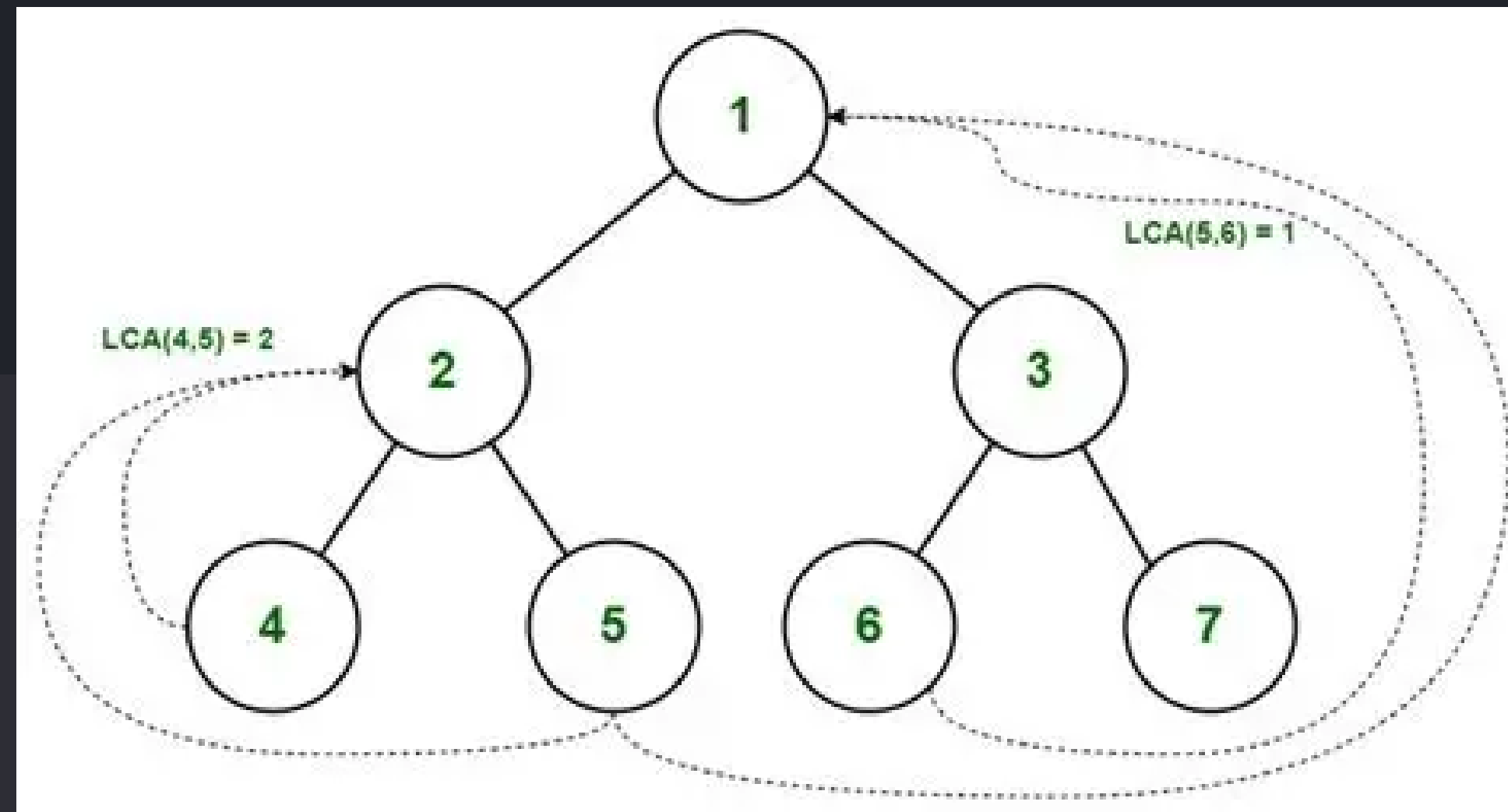
Contenidos



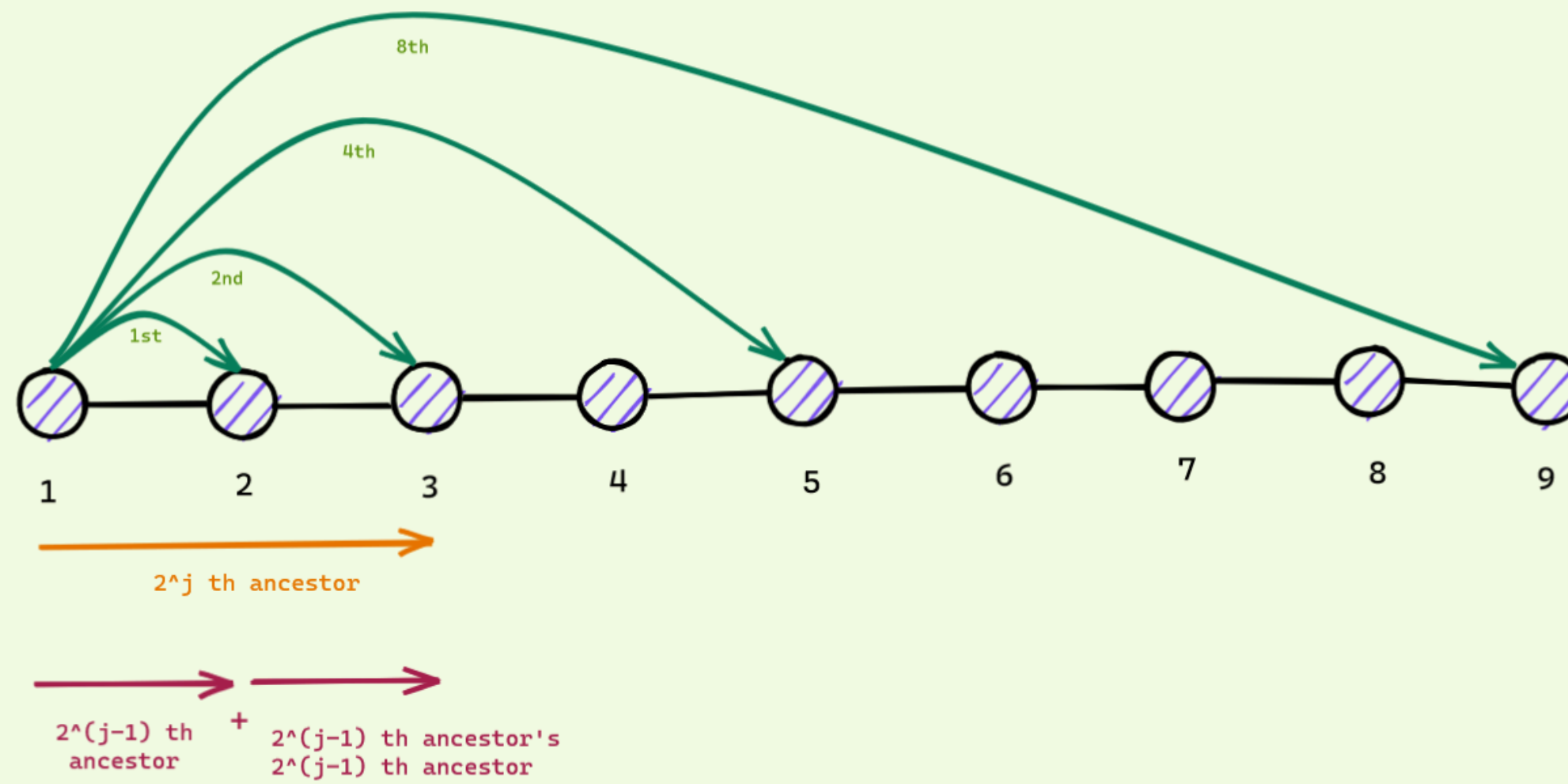
- 01 Introducción
- 02 Objetivos
- 03 Código
- 04 Visualización
- 05 Complejidades
- 06 Aplicaciones reales
- 07 Conclusión

Introducción {

Algoritmo principal con fuerza bruta



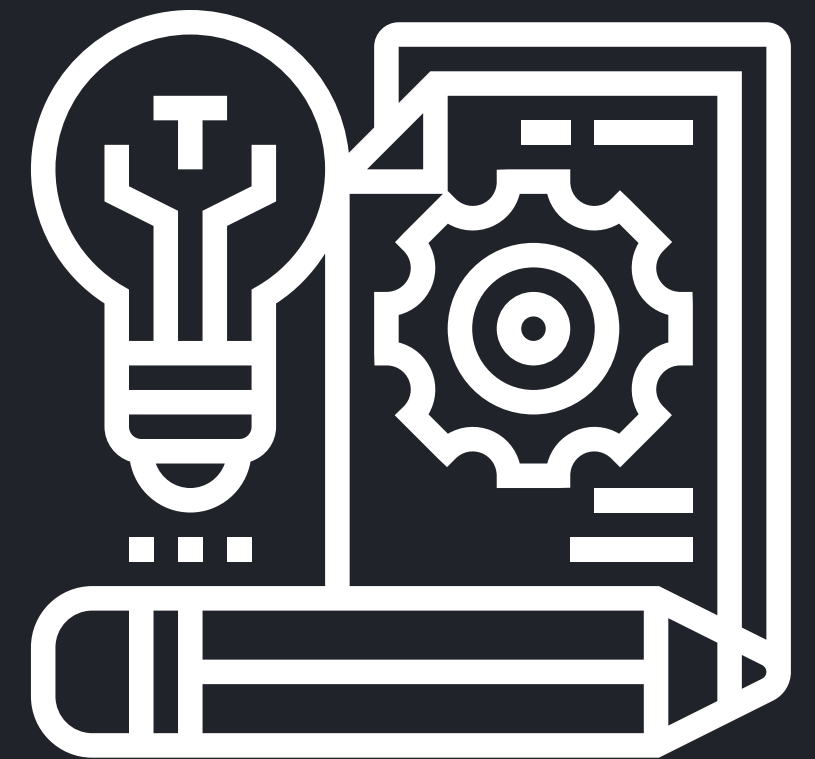
Binary Lifting



Objetivos {

- **Modificar algoritmo LCA**
- **Aplicar divide y vencerás**
- **Visualizar recorrido de los algoritmos**
- **Mostrar eficiencia y complejidad**
- **Utilidad real**

}



Código {

1

```
# Definición del Árbol
self.num_nodos = 12
self.aristas = [(1, 2), (1, 3), (2, 4), (2, 5), (3, 6), (5, 7), (5, 8),
                (8, 9), (8, 10), (10, 11), (11, 12)]
self.coordenadas = {
    1: (450, 50), 2: (300, 120), 3: (600, 120), 4: (225, 190),
    5: (375, 190), 6: (600, 190), 7: (325, 260), 8: (425, 260),
    9: (475, 330), 10: (375, 330), 11: (375, 400), 12: (375, 470)
}
```

2

```
def construir_lista_adyacencia(self): 1 usage
    for u, v in self.aristas:
        self.lista_adyacencia[u].append(v)
        self.lista_adyacencia[v].append(u)
```

3

```
def preprocesar_algoritmos(self): # Ejecuta las fases de preprocesamiento
    # Deep First Search (DFS) para niveles y padres directos
    stack = [(1, 0, 1)]
    visitados = {1}
    while stack:
        u, p, l = stack.pop()
        self.nivel[u] = l
        self.padre[u][0] = p
        for v in self.lista_adyacencia[u]:
            if v != p and v not in visitados:
                visitados.add(v)
                stack.append((v, u, l + 1))
    # Tabla de saltos (solo para Binary Lifting)
    for i in range(1, self.log_max + 1):
        for u in range(1, self.num_nodos + 1):
            ancestro_intermedio = self.padre[u][i - 1]
            if ancestro_intermedio != 0:
                self.padre[u][i] = self.padre[ancestro_intermedio][i - 1]
```

4

```
def obtener_lca(self, u, v): 1 usage  ⚡ Angel Villalvazo +1 *
    if self.nivel[u] > self.nivel[v]:
        u, v = v, u
    for i in range(self.log_max, -1, -1):
        if self.nivel[v] - (1 << i) >= self.nivel[u]:
            v = self.padre[v][i]
    if u == v:
        return u
    for i in range(self.log_max, -1, -1):
        if self.padre[u][i] != 0 and self.padre[u][i] != self.padre[v][i]:
            u = self.padre[u][i]
            v = self.padre[v][i]
    return self.padre[u][0]
```

5

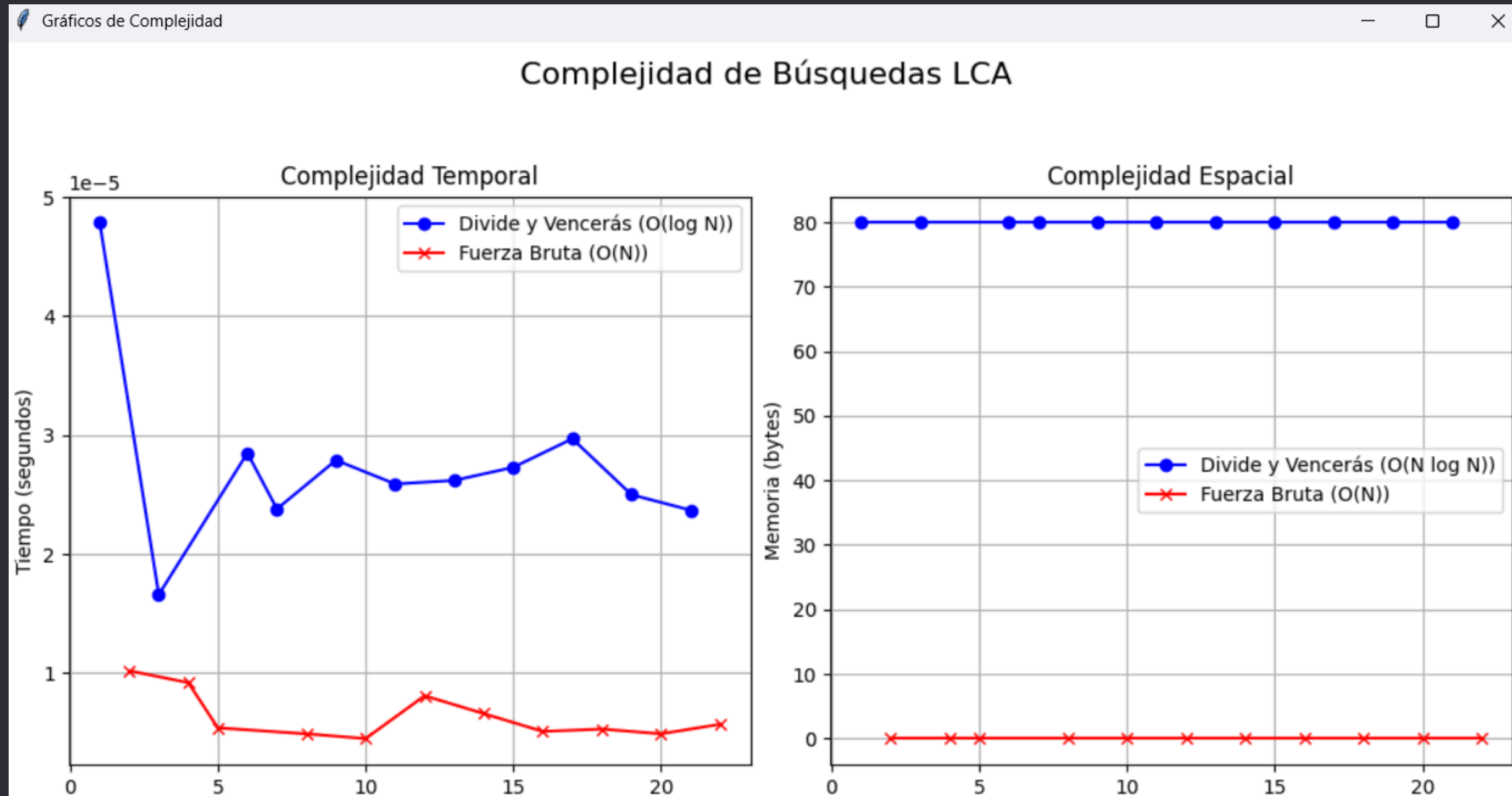
Implementación con
fuerza bruta

```
def obtener_lca_fuerza_bruta(self, u, v): 1 usage  ⚡ Angel Villalvazo *
    while self.nivel[u] > self.nivel[v]:
        u = self.padre[u][0]
    while self.nivel[v] > self.nivel[u]:
        v = self.padre[v][0]
    while u != v:
        u = self.padre[u][0]
        v = self.padre[v][0]
    return u
```

}

7

Complejidad Temporal Y Espacial {



}

Aplicaciones en la vida real {

Control de Versiones

Modela el historial de un proyecto como un Grafo Acíclico Dirigido (DAG), donde cada commit es un nodo y las relaciones padre-hijo conectan los commits a lo largo del tiempo.



Redes De Computadoras

Los protocolos de enrutamiento como OSPF que permite a los routers intercambiar información sobre el estado de la red y calcular las rutas más eficientes para el tráfico de datos están diseñados para tomar redes caóticas y crear un árbol lógico y estable para enviar datos.

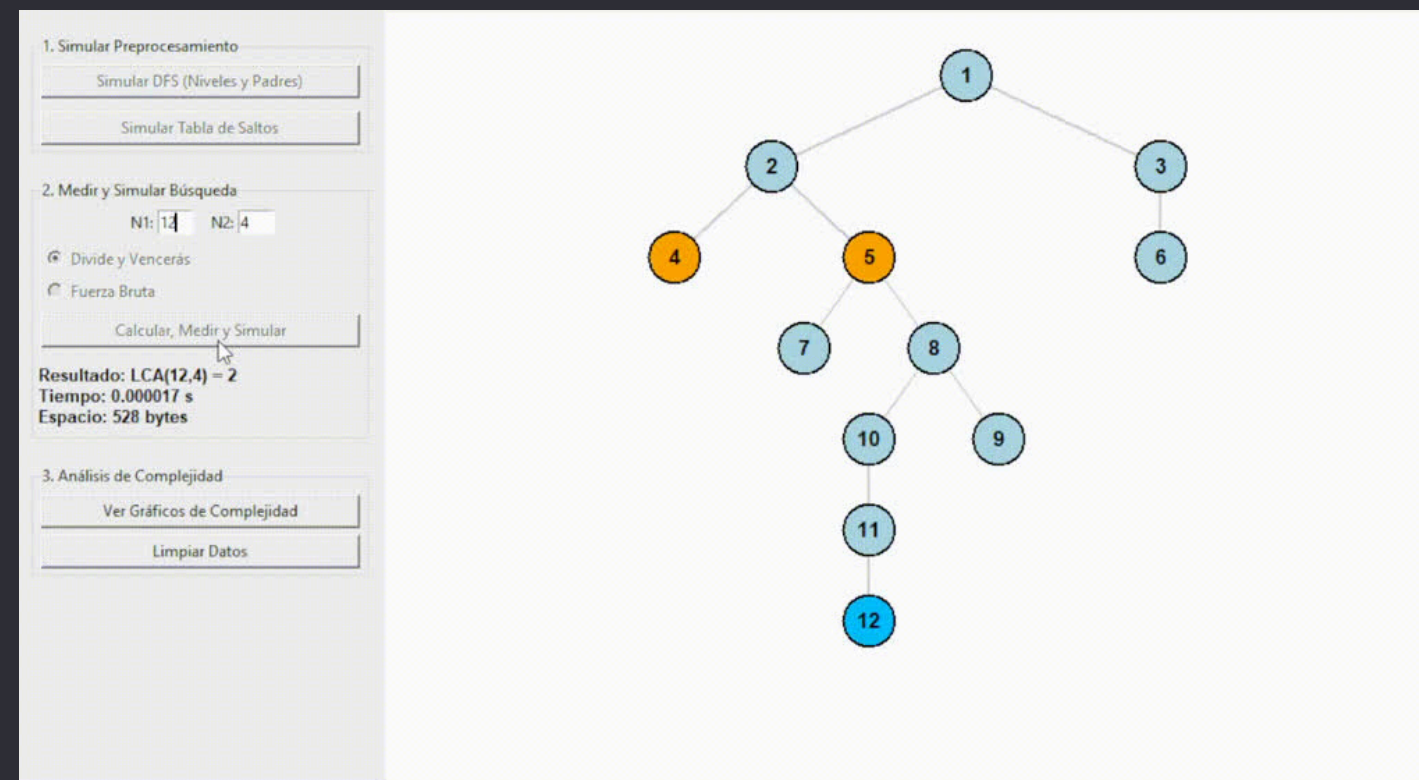


Conclusión {



Algoritmo

Con esta práctica demostramos la diferencia de eficiencia entre los métodos que puede tener un algoritmo en tiempo real.



Limitaciones

- Solamente es viable con árboles estáticos.
- Coste de preprocesamiento de $O(N \log N)$.
- Complejidad de implementación.

```
<!--Fin de la presentación-->
```

Gracias {

```
<Algoritmo="Lowest Common Ancestor  
con Binary Lifting"/>
```

}